Modern Distributed Systems Architecture

Chapter 1: Principles of Fault-Tolerant State Machines

Distributed systems are composed of autonomous computing entities that communicate over asynchronous networks. Network partitions, packet loss, and unpredictable delays are normal operating conditions rather than exceptional failures.

Key Architectural Pillars:
1. Consistency, Availability, Partition Tolerance (CAP Theorem)
2. State Machine Replication via Consensus (Raft / Multi-Paxos)
3. At-least-once message delivery with idempotent execution receivers
4. Distributed tracing and telemetry correlation IDs

Raft Consensus Protocol & Heartbeats

Chapter 2: Leader Election and Invariants

A Raft cluster consists of several servers; typically 5, allowing the system to tolerate 2 failures. At any given time, each server is in one of three states:

- Leader: Handles all client requests, replicates log entries.
- Follower: Passive, responds to requests from leaders and candidates.
- Candidate: Used to elect a new leader during election timeouts.

AppendEntries RPC Invariants:

- If two entries in different logs have the same index and term, they store the same command.
- If two entries in different logs have the same index and term, then their logs are identical in all preceding entries.

Code Focus Mode: Distributed Lock Implementation

Chapter 3: Lease Management & Fencing Tokens

```
// Distributed Lock with TTL and Fencing Token
export class DistributedLock {
  private readonly key: string;
  private readonly leaseMs: number;
  private fencingToken: number = 0;

  async acquire(): Promise<{ token: number; release: () => Promise<void> }> {
    const token = await redis.eval(ACQUIRE_LUA_SCRIPT, [this.key], [this.leaseMs]);
    if (!token) throw new Error("Could not acquire distributed lock");
    this.fencingToken = Number(token);
    return { token: this.fencingToken, release: () => this.release() };
  }
}
```

Saga Pattern vs Two-Phase Commit

Chapter 4: Orchestration and Compensating Transactions

In microservice architectures where data is partitioned across multiple distinct databases, atomic 2PC protocols impose high latency and single points of failure (the coordinator).

The Saga pattern breaks a distributed transaction into a sequence of local transactions. Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga.

If a local transaction fails, the saga executes compensating transactions that undo the changes made by preceding transactions.

High-Throughput Storage & Backup Architecture

Chapter 5: Telegram Cloud Persistence & Deduplication

Integrating Telegram Bot API as an encrypted cloud storage tier enables:
- Zero-cost decentralized document archiving for verified open-access libraries.
- High network throughput and global edge CDN availability.
- Automatic SHA-256 integrity verification and content-addressable storage keys.

Upload constraints & architecture:
- Telegram Bot API standard file upload threshold: 50 MB.
- Self-hosted local Telegram Bot API server threshold: 2,000 MB (2 GB).
- Application level validation verifies magic bytes %PDF- and checks EOF integrity.